

SPRAWOZDANIE

Tytuł	Implementacja Algorytmu Roju Cząstek (PSO) w języku C
Data	22.01.2026
Wykonali	Milena Piasecka, Ewa Kułaga
Sprawdzili	mgr inż. Paweł Kluge mgr inż. Aleksander Zamojski

Spis treści

CEL PROJEKTU	2
ZASADA DZIAŁANIA ALGORYTMU PSO	2
OPIS ZADANIA.....	2
OPIS PODZIAŁU PRAC W ZESPOLE	2
OPIS IMPLEMENTACJI ZADANIA	2
SPOSÓB WYWOŁANIA PROGRAMU	4
TESTY DZIAŁANIA	5
WNIOSKI.....	6

Cel projektu

Celem projektu było wykorzystanie algorytmu PSO do napisania symulatora „Misji Ratunkowej”. Program miał symulować rój dronów, które zostały losowo rozrzucone na mapie i ich zadaniem było najszybsze zlokalizowanie źródła sygnału, którym był turysta.

Zasada działania algorytmu PSO

PSO(Particle Swarm Optimization) to algorytm optymalizacyjny inspirowany zachowaniem stad ptaków lub ławic ryb. Iteracyjnie poszukuje najlepszego rozwiązania, zarządzając populacją(rojem), gdzie ruch każdej cząstki określany jest przez bezwładność – cząstka chce utrzymać swój dotychczasowy kierunek ruchu - oraz dostosowuje swoją prędkość i pozycję na podstawie pamięci własnej - najlepszego miejsca, w którym była - i wiedzy całego roju – najlepszego miejsca znalezione przez jakąkolwiek cząstkę.

Opis zadania

Użytkownik przekazuje programowi odpowiednie parametry. Następnie konfigurowana jest mapa o wymiarach i wartościach podanych przez użytkownika. Podobnie dzieje się w przypadku roju. Korzystając z algorytmu PSO szukana jest największa wartość cząstki i roju poprzez zmianę pozycji i prędkości. Operacje wykonują się w pętli iterującej zgodnie z ilością pobraną od użytkownika dla każdej z cząstek. Po symulacji wyświetlane zostaje rozwiązanie, czyli najlepsze dotychczasowe wartości znalezione przez rój podczas aktualizacji wartości.

Opis podziału prac w zespole

Projekt składał się z plików o nazwach: main.c, pso.c, logger.c, utils.c, map.c. Milena napisała pliki: pso.c, utils.c, a Ewa – main.c, logger.c i map.c

Opis implementacji zadania

Program został podzielony na pliki pomocnicze, w celu zwiększenia przejrzystości kodu oraz lepszego podziału pracy. Pliki z rozszerzeniem .c zostały wyposażone w bibliotekę .h, aby oddzielić deklarację od definicji funkcji, co również wpłynęło na czytelność kodu.

```
typedef struct {
    double x, y;
    double vx, vy;
    double best_x, best_y;
    double best_value;
} Particle;
```

Zrzut ekranu 1

Cząstka została opisana za pomocą struktury, aby łatwiej przedstawić wszystkie jej parametry. Każda z nich składa się ze współrzędnych (x,y), prędkości (vx, vy), najlepszej dotąd pozycji danej cząstki (best_x, best_y) i najlepszej dotąd wartości (best_value).

```
typedef struct {
    Particle *particles;
    int size;

    double gbest_x, gbest_y;
    double gbest_value;

    double w, c1, c2;
} Swarm;
```

Zrzut ekranu 2

Rój składa się ze wskaźnika na cząstkę, aby przypisać jej związek z populacją. Struktura zawiera: liczbę cząstek (size), najlepsze znalezione dotąd współrzędne w roju (gbest_x, gbest_y) oraz parametry (w, c1, c2) określone w pliku konfiguracyjnym lub domyślnie kolejno: 0.5, 1.0 i 1.0.

```
typedef struct {
    int szerokosc;
    int wysokosc;
    double **value;
}map;
```

Zrzut ekranu 3

Struktura mapy składa się z parametrów, które są określone w pliku mapy.

```
Swarm* init_swarm(int size, map *mapp) {
    Swarm *s = malloc(sizeof(Swarm));
    s->size = size;
    s->particles = malloc(size * sizeof(Particle));
```

Zrzut ekranu 4

Do liczby cząstek została dynamicznie zaalokowana pamięć korzystając z funkcji malloc z biblioteki stdlib.h.

```
p->vx = s->w * p->vx +
        s->c1 * r1 * (p->best_x - p->x) +
        s->c2 * r2 * (s->gbest_x - p->x);

p->vy = s->w * p->vy +
        s->c1 * r1 * (p->best_y - p->y) +
        s->c2 * r2 * (s->gbest_y - p->y);
```

Zrzut ekranu 5

Prędkości vx i vy są obliczane za pomocą wzoru:

$$v_i(t+1) = w * v_i(t) + c_1 r_1 * (pBest_i - x_i(t)) + c_2 r_2 * (gBest - x_i(t))$$

gdzie:

w – waga bezwładności

v – prędkość

x – współrzędna x-owa

c₁ i c₂ – współczynniki uczenia

r₁ i r₂ - losowe liczby

pBest! – najlepsza znana dotychczasowa pozycja cząstki

gBest – najlepsza znana dotychczasowa pozycja całego roju

i – numer i-tej cząstki

t – iteracja

Sposób wywołania programu

Plik Makefile automatyzuje proces budowania programu PSO, zarządzając zależnościami pomiędzy plikami źródłowymi. Skrypt wykorzystuje kompilator gcc z flagami -Wall i -Wextra w celu wykrywania błędów oraz -O2, która przyspiesza działanie gotowego programu bez zwiększania rozmiaru kodu. Po wpisaniu komendy make, program spróbuje zbudować cel o nazwie pso. Reguła clean zawarta na samym dole pliku pozwala na przywrócenie projektu do stanu początkowego.

```

M Makefile
1  CC = gcc
2  CFLAGS = -Wall -Wextra -O2
3
4  OBJ = main.o map.o pso.o utils.o logger.o
5
6  all:pso
7
8  pso: $(OBJ)
9      $(CC) $(CFLAGS) $(OBJ) -o pso
10
11 main.o: main.c map.h pso.h logger.h utils.h
12     $(CC) $(CFLAGS) -c main.c
13
14 map.o: map.c map.h
15     $(CC) $(CFLAGS) -c map.c
16
17 pso.o: pso.c pso.h map.h utils.h
18     $(CC) $(CFLAGS) -c pso.c
19
20 utils.o: utils.c utils.h
21     $(CC) $(CFLAGS) -c utils.c
22
23 logger.o: logger.c logger.h pso.h
24     $(CC) $(CFLAGS) -c logger.c
25
26 clean:
27     rm -f *.o pso
28

```

Zrzut ekranu 6

Program można wywołać w terminalu za pomocą komendy:

```
./pso <plik_mapy> -p <liczba_czastek> -i <liczba_iteracji> -c <plik_konfiguracyjny z
parametrami_PSO> -n <co_ktora_iteracje_zapis_postepow>
```

W przypadku zbyt małej liczby argumentów program poinformuje o błędzie i zakończy działania. Jeśli użytkownik nie poda parametrów zostaną użyte te domyślne. Przykładowe wywołanie `./pso terrain.txt -p 50 -i 200 -c config_file -n 2` oznacza, że użytkownik przekazuje programowi, że mapa znajduje się w pliku `terrain.txt`, rój składa się z 50 cząstek, symulacja wykona się 200 razy, plik konfiguracyjny o nazwie `config_file` z parametrami oraz chce, aby zapisywać postępy co 2 iteracje.

Testy działania

Działanie programu dla pliku konfiguracyjnego `config1.txt` o parametrach: `0.8 1.5 1.5` oraz dla mapy zawartej w pliku `mapa.txt` o wartościach:

```

7 7
-1000.00 -1000.00 -1000.00 -1000.00 24.63 25.05 25.20
-1000.00 -1000.00 -1000.00 -1000.00 7.70 26.21 26.40
-1000.00 -1000.00 -1000.00 -1000.00 7.98 8.20 27.60
-1000.00 -1000.00 -1000.00 -1000.00 -1000.00 -1000.00 -1000.00
2.05 2.16 2.27 -1000.00 -1000.00 -1000.00 -1000.00

```

```
6.81 2.15 2.25 -1000.00 -1000.00 -1000.00 -1000.00
6.72 7.06 2.22 -1000.00 -1000.00 -1000.00 -1000.00
```

```
● → Projekt-koncowy git:(nowa) x ./pso dane/mapa.txt -p 50 -i 200 -c dane/config1.txt -n 2
Mapa: dane/mapa.txt
Konfiguracja roju: liczba czastek = 50, liczba iteracji = 2, logowanie = 0
Parametry PSO wczytane z pliku: w=0.80, c1=1.50, c2=1.50
Wyniki – najlepsze rozwiazanie:
x = 6.883
y = 2.861
wartość = 27.600
```

Zrzut ekranu 7

Wnioski: Algorytm skutecznie zadziałał i znalazł globalne maksimum o wartości 27.60, nie dając się uwieźć w obszarach o niższym sygnale, co świadczy o dobrej zdolności do eksploracji przy tych ustawieniach.

Działanie programu dla braku pliku konfiguracyjnego oraz dla mapy zawartej w pliku mapa2.txt o wartościach:

```
5 5
1.00 2.00 3.00 4.00 5.00
2.00 3.00 4.00 5.00 6.00
3.00 4.00 5.00 6.00 7.00
4.00 5.00 6.00 7.00 8.00
5.00 6.00 7.00 8.00 100.00
```

```
● → Projekt-koncowy git:(nowa) x ./pso dane/mapa2.txt
Mapa: dane/mapa2.txt
Konfiguracja roju: liczba czastek = 30, liczba iteracji = 100, logowanie = 0
Wyniki – najlepsze rozwiazanie:
x = 4.997
y = 4.443
wartość = 100.000
```

Zrzut ekranu 8

Wnioski: Program zadziałał poprawnie – bez uwzględnienia pliku konfiguracyjnego program przypisał wartości domyślne. Test pokazuje że program nie ma problemu z znajdowaniem wartości w narożnikach mapy.

Wnioski

Projekt pozwolił nam na zapoznanie się z algorytmem PSO poprzez pisanie kodu rozwiązującego zadanie „Misji Ratunkowej”. Mogliśmy wykorzystać wiedzę na temat języka C, którą zdobyliśmy w trakcie semestru w praktyce.